

LIQPLAT Process Evaluation: Quantitative Analysis Plan

Johannes M. Schwenke

2025-09-25

Introduction

This document outlines the planned quantitative analyses for the process evaluation embedded within the LIQPLAT trial. The objective of these analyses is to descriptively explore and quantify key implementation processes and mechanisms.

All analyses will be conducted within a Bayesian framework to quantify uncertainty. We will primarily use the `brms` and `rstanarm` packages for modeling and the `marginalEffects` package for prediction, comparison, and visualization of model outputs.

Here we use a synthetic dataset for demonstration purposes. This dummy data mimics the expected structure of the final trial data in form only. It is not internally consistent and does not reflect real-world distributions or associations. The actual analysis will be performed on the final trial dataset following the same analysis plan. Any changes to the analyses are version controlled.

```
# Load packages
library(brms)
library(rstanarm)
library(marginalEffects)
library(tidyverse)
library(tidybayes)
library(patchwork)
library(splines)

# Set seed for reproducibility
set.seed(1234)

# --- Generate Synthetic Data ---
n_total_screened <- 1000
n_invited <- 180
n_control <- 90
n_ineligible <- n_total_screened - (n_invited + n_control)

sim_data <- tibble(
  patient_id = 1:n_total_screened,
  age = rnorm(n_total_screened, 65, 10),
  cancer_type = factor(sample(c("Lung", "Colorectal", "Head and Neck", "Other"),
    n_total_screened, replace = TRUE)),
  eligible = c(rep(1, n_invited + n_control), rep(0, n_ineligible)))

group_assignments <- character(n_total_screened)
is_eligible <- sim_data$eligible == 1
group_assignments[!is_eligible] <- "Ineligible"
```

```

group_assignments[is_eligible] <- sample(c(rep("SAT", n_invited), rep("Control",
n_control)))

sim_data <- sim_data |>
  mutate(
    # Add the pre-defined group assignments
    group = factor(group_assignments, levels = c("SAT", "Control", "Ineligible")),

    # Process outcomes based on group
    offered_invitation = rbinom(n(), 1, ifelse(group == "SAT", 0.95, 0)),
    accepted_invitation = rbinom(n(), 1, ifelse(offered_invitation == 1, 0.90, 0)),

    # ctDNA outcomes (only for the SAT group)
    ctdna_error = ifelse(group == "SAT", rbinom(n(), 1, 0.05), NA),
    ctdna_detected_baseline = ifelse(group == "SAT", rbinom(n(), 1, 0.75), NA),
    chip_suspicion = ifelse(group == "SAT" & ctdna_detected_baseline == 1, rbinom(n(),
1, 0.20), NA),
    actionable_alteration = ifelse(group == "SAT", rbinom(n(), 1, 0.20), NA),

    # Follow-up and survival endpoints for all patients
    follow_up_time = 12, # months
    time_to_switch = rexp(n(), rate = 1/12),
    event_switch = if_else(time_to_switch < follow_up_time, 1, 0),
    # time to palliative care is of course not independent of time to switch, but for
simplicity we simulate independently here
    time_to_bsc = rexp(n(), rate = 1/18),
    event_bsc = if_else(time_to_bsc < follow_up_time, 1, 0),

    targeted_therapy = rbinom(n(), 1, prob = case_when(
      group == "SAT" ~ 0.1,
      group == "Control" ~ 0.05,
      TRUE ~ 0.05 # For ineligible group
    )),
    n_blood_products = rnbinom(n(), size = 1, mu = 1.5)
  ) |>
  rename(grp = group)

```

Patient Eligibility

To quantify the proportion of patients screened for the trial who were deemed eligible for participation.

```

eligible_summary <- sim_data |>
  distinct(patient_id, eligible) |>
  count(eligible) |>
  mutate(prop = n / sum(n))
eligible_summary

```

```

# A tibble: 2 × 3
  eligible      n prop
  <dbl> <int> <dbl>
1       0   730  0.73
2       1   270  0.27

```

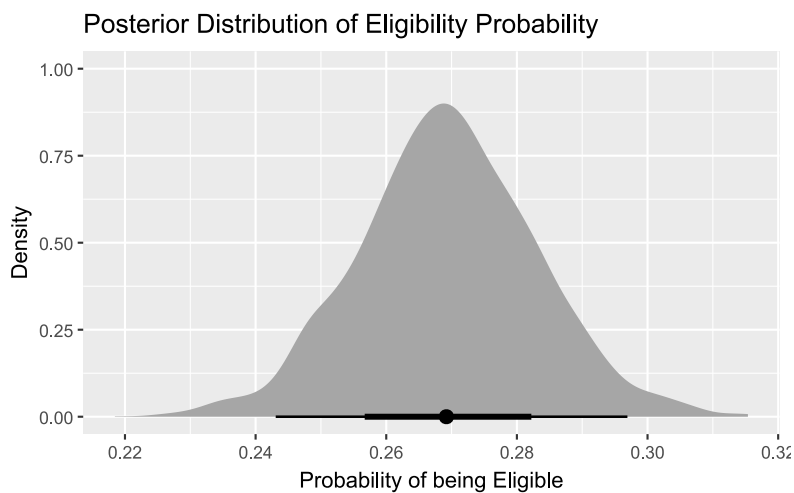
```

fit_eligible <- brm(
  eligible ~ 1,
  data = sim_data,
  family = bernoulli(),
  backend = "cmdstanr",
  iter = 2000,
  chains = 4,
  cores = 1,
  seed = 123,
  file = "9-process-evaluation/models/fit_eligible.rds",
  file_refit = "on_change"
)

pred_eligible <- avg_predictions(fit_eligible)

pred_eligible |>
  posterior_draws() |>
  ggplot(aes(x = draw)) +
  stat_halfeye() +
  labs(
    title = "Posterior Distribution of Eligibility Probability",
    x = "Probability of being Eligible", y = "Density"
  )

```



Invitation Offered by Physician

To quantify the proportion of patients who were scheduled for invitation and were subsequently offered participation by their physician.

```

offered_summary <- sim_data |>
  filter(grp == "SAT") |>
  count(offered_invitation) |>
  mutate(prop = n / sum(n))

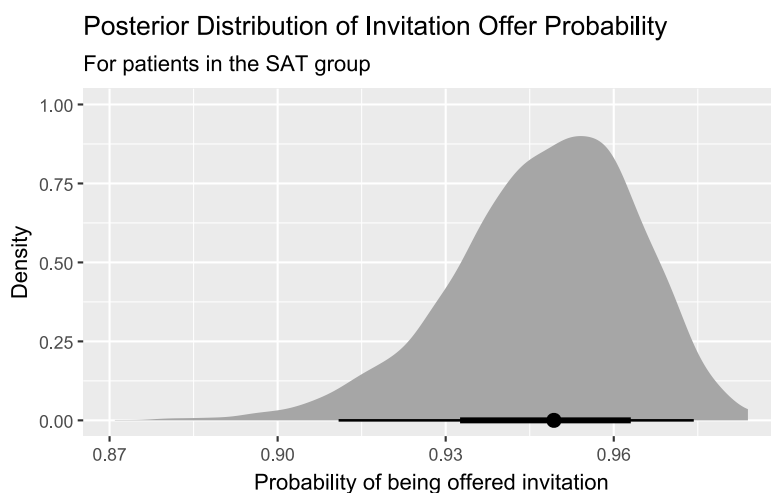
offered_summary

```

```
# A tibble: 2 × 3
  offered_invitation    n prop
      <int> <int> <dbl>
1           0         9  0.05
2           1        171  0.95
```

```
fit_offered <- brm(
  offered_invitation ~ 1,
  data = sim_data |> filter(grp == "SAT"),
  family = bernoulli(),
  iter = 2000,
  chains = 4,
  cores = 1,
  seed = 123,
  backend = "cmdstanr",
  file = "9-process-evaluation/models/fit_offered.rds",
  file_refit = "on_change"
)

avg_predictions(fit_offered) |>
  posterior_draws() |>
  ggplot(aes(x = draw)) +
  stat_halfeye() +
  labs(
    title = "Posterior Distribution of Invitation Offer Probability",
    subtitle = "For patients in the SAT group",
    x = "Probability of being offered invitation", y = "Density"
  )
```



3. Patient Acceptance of Invitation

To quantify the proportion of patients who, after being offered participation, provided informed consent to join the trial.

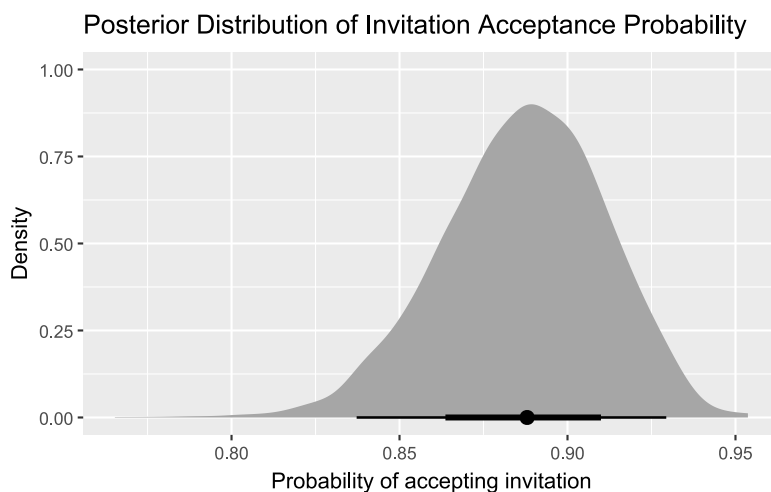
```
accepted_summary <- sim_data |>
  filter(offered_invitation == 1) |>
  count(accepted_invitation) |>
```

```
mutate(prop = n / sum(n))
accepted_summary
```

```
# A tibble: 2 × 3
  accepted_invitation    n prop
      <int> <int> <dbl>
1           0      19 0.111
2           1     152 0.889
```

```
fit_accepted <- brm(
  accepted_invitation ~ 1,
  data = sim_data |> filter(offered_invitation == 1),
  family = bernoulli(),
  backend = "cmdstanr",
  iter = 2000,
  chains = 4,
  cores = 1,
  seed = 123,
  file = "9-process-evaluation/models/fit_accepted.rds",
  file_refit = "on_change"
)

avg_predictions(fit_accepted) |>
  posterior_draws() |>
  ggplot(aes(x = draw)) +
  stat_halfeye() +
  labs(
    title = "Posterior Distribution of Invitation Acceptance Probability",
    x = "Probability of accepting invitation", y = "Density"
  )
```



4. Technical Feasibility of ctDNA Analysis

To evaluate three key aspects of technical feasibility: (1) the proportion of ctDNA samples processed without error; (2) the proportion of baseline samples with detectable ctDNA; and (3) the proportion of detected variants suspected to be from Clonal Hematopoiesis of Indeterminate Potential (CHIP).

```

# 4.1 Proportion of ctDNA samples analyzed without error
set.seed(42)

# Create data for 150 patients, each providing 1 to 4 samples (or more in actuality in
LIQPLAT)
sim_data_samples_error <- tibble(
  patient_id = 1:150
) |>
# Each patient has a random number of samples (1 to 4)
mutate(num_samples = sample(1:4, size = n(), replace = TRUE)) |>
uncount(num_samples) |>
# Add a patient-specific random effect (some patients' samples may be
# more prone to errors for biological reasons)
group_by(patient_id) |>
mutate(patient_effect_error = rnorm(1, mean = 0, sd = 1.0)) |>
ungroup() |>
# Calculate the final probability of an error for each sample
mutate(
  prob_error = plogis(-4 + patient_effect_error), # Low base error rate
  ctdna_error = rbinom(n(), 1, prob = prob_error)
)

fit_ctdna_error <- brm(
  ctdna_error ~ 1 + (1 | patient_id),
  data = sim_data_samples_error,
  family = bernoulli(),
  iter = 2000, chains = 4, cores = 4, seed = 123,
  file = "9-process-evaluation/models/fit_ctdna_error.rds",
  file_refit = "on_change"
)

# 4.2 Proportion of ctDNA-analyses detecting ctDNA at baseline
fit_ctdna_detected <- brm(
  ctdna_detected_baseline ~ 1,
  data = sim_data |> filter(accepted_invitation == 1),
  family = bernoulli(),
  iter = 2000, chains = 4, cores = 1, seed = 123,
  file = "9-process-evaluation/models/fit_ctdna_detected.rds",
  file_refit = "on_change"
)

# 4.3 Proportion of variants found on ctDNA-analyses being labelled with suspicion of
CHIP
# We have multiple variants per patient, which could be CHIP, and multiple samples per
patient.

set.seed(42)
gene_list <- c(
  paste0("CHIP_", 1:5), # High probability of being CHIP
  paste0("TUMOR_", 1:15) # Low probability of being CHIP
)

patient_data <- tibble(
  patient_id = 1:50,
  age = rnorm(50, mean = 65, sd = 10)/10, # Age in decades

```

```

    patient_effect = rnorm(50, mean = 0, sd = 0.5)
  )

sim_data_samples <- tidyr::crossing(
  patient_id = patient_data$patient_id,
  time_point = c(0, 3, 6) # Months
) |>
left_join(patient_data, by = "patient_id") |>
mutate(
  total_variants = rpois(n(), lambda = 3)
) |>
filter(total_variants > 0)

beta_age <- 0.5 # increase in log-odds per decade of age

sim_data_variants <- sim_data_samples |>
  uncount(total_variants) |>
  group_by(patient_id, time_point) |>
  mutate(
    # Assign a random gene ID to each row
    gene_id = sample(gene_list, n(), replace = TRUE),
    # The probability of being CHIP depends on the gene type
    gene_effect = ifelse(startsWith(gene_id, "CHIP"), 1.5, -1.5),
    prob_chip_variant = plogis(-4 + patient_effect + gene_effect + beta_age * age),
    is_chip_suspicion = rbinom(n(), 1, prob = prob_chip_variant)
  ) |>
  ungroup()

fit_chip <- brm(
  is_chip_suspicion ~ 1 + ns(age, df = 2) + time_point + (1 | patient_id) + (1 |
gene_id),
  data = sim_data_variants,
  family = bernoulli(),
  iter = 2000,
  chains = 4,
  cores = 4,
  seed = 123,
  file = "9-process-evaluation/models/fit_chip.rds",
  file_refit = "on_change"
)

# Plotting the posteriors
p1 <- avg_predictions(fit_ctdna_error, re_formula = NULL) |>
  posterior_draws() |>
  ggplot(aes(x = 1 - draw)) + stat_halfeye() + labs(title = "Proportion without Error",
x = "Probability")

p2 <- avg_predictions(fit_ctdna_detected) |>
  posterior_draws() |>
  ggplot(aes(x = draw)) + stat_halfeye() + labs(title = "Proportion with ctDNA Detected
at Baseline", x = "Probability")

p3 <- avg_predictions(
  fit_chip,
  re_formula = NULL) |> # NA ignores random effects, NULL marginalizes over them

```

```

posterior_draws() |>
ggplot(aes(x = draw)) + stat_halfeye() + labs(title = "Estimated (marginal)
Probability of a found Variant to be CHIP", x = "Probability")

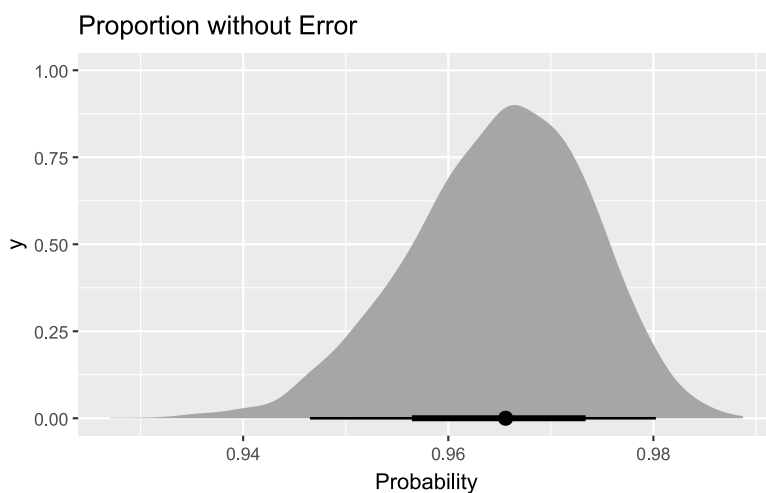
# we marginalize over the covariates to look just at the effect of age, alls things
equal
p4 <- plot_predictions(
  fit_chip,
  by = "age",
  variable = "age",
  re_formula = NULL # This marginalized voer random effects
) +
  labs(
    title = "Predicted Probability of a Variant being CHIP by Age",
    x = "Age (Decades)",
    y = "Probability"
  )

# marginalized probability for each gene to be classified as CHIP, intergrates random
effects
variant_specific <- avg_predictions(
  fit_chip,
  by = "gene_id",
  variables = "gene_id",
  re_formula = NULL
)

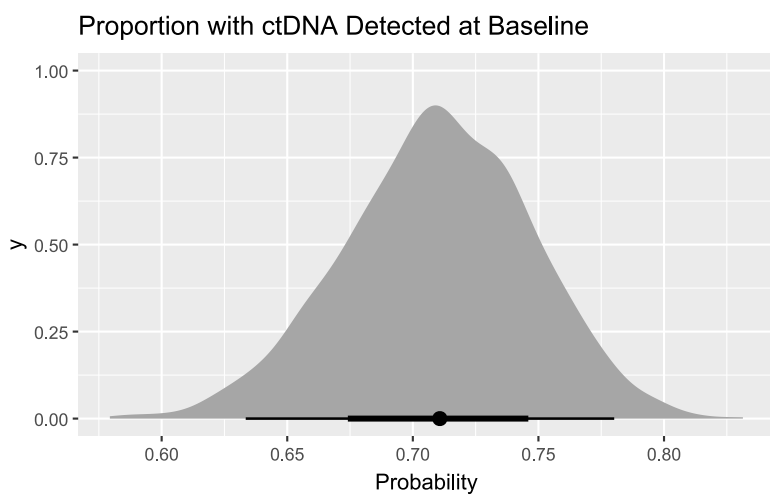
p5 <- variant_specific |>
posterior_draws() |>
ggplot(aes(x = draw, y = gene_id)) +
ggdist::stat_halfeye() +
  labs(
    title = "Posterior of Probability of a Variant being CHIP by Gene",
    x = "Probability",
    y = "Gene"
  )

p1

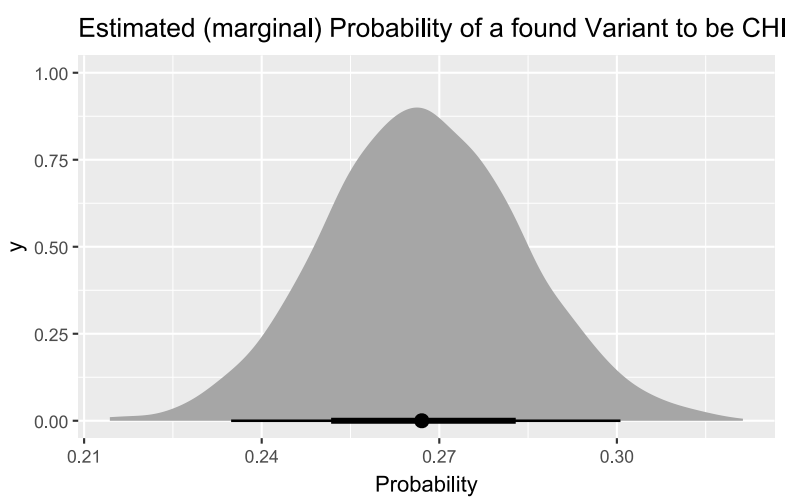
```



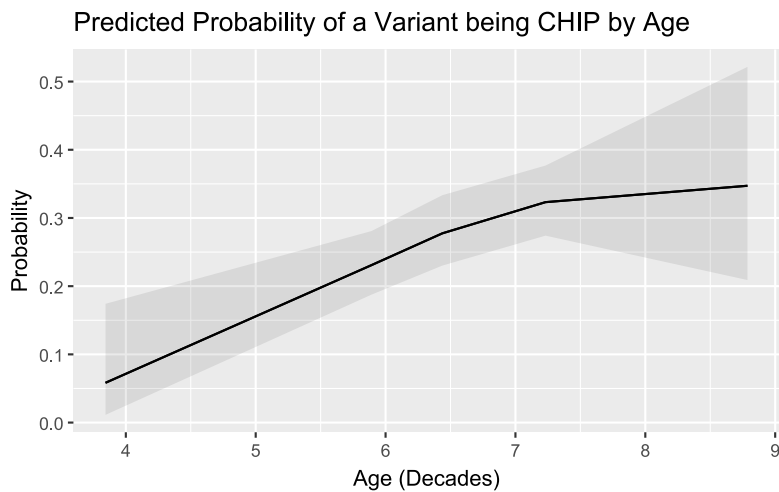
p2



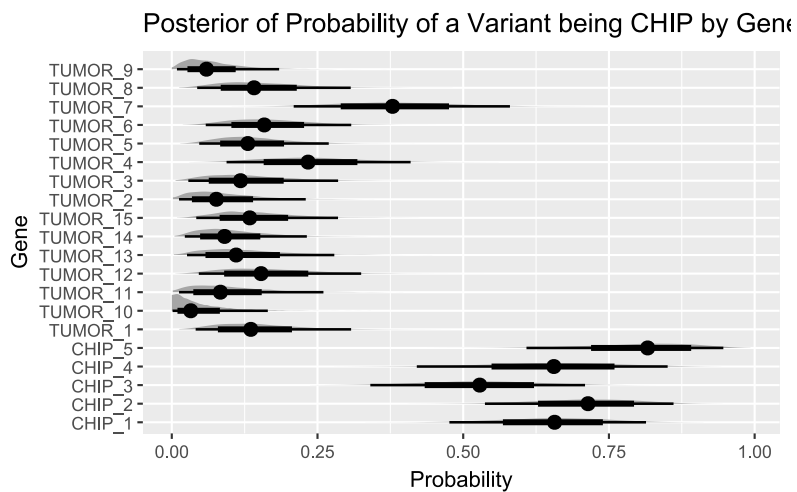
p3



p4



p5



5. Time to Treatment Switch or Stop or Death

To compare the time from treatment initiation to the composite endpoint of treatment switch, stop, or death between the SAT and external control groups.

```
surv_data <- sim_data |>
  filter(grp %in% c("SAT", "Control")) |>
  mutate(time_to_switch = pmin(time_to_switch, follow_up_time),
         event_switch = ifelse(time_to_switch < follow_up_time, 1, 0),
         grp = factor(grp, levels = c("SAT", "Control")))

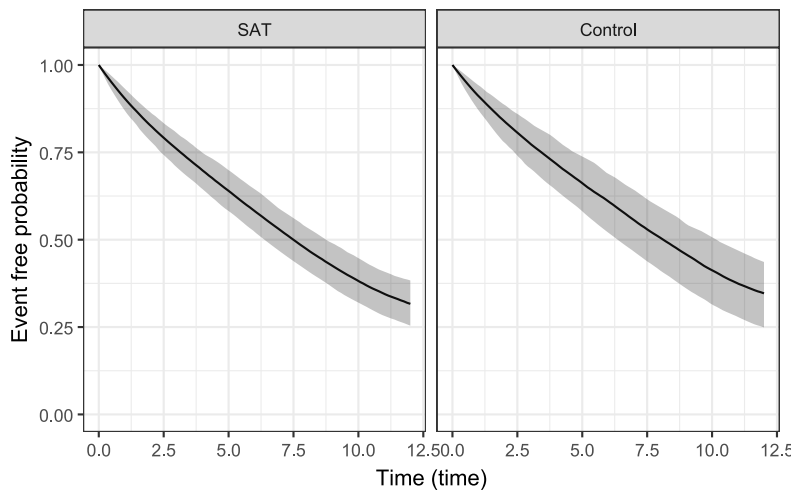
fit_surv_switch <- stan_surv(
  formula = Surv(time_to_switch, event_switch) ~ grp,
  data = surv_data,
  basehaz = "ms",
  basehaz_ops = list(df = 5),
  iter = 2000,
  chains = 4,
  cores = 4,
```

```
seed = 123
)
```

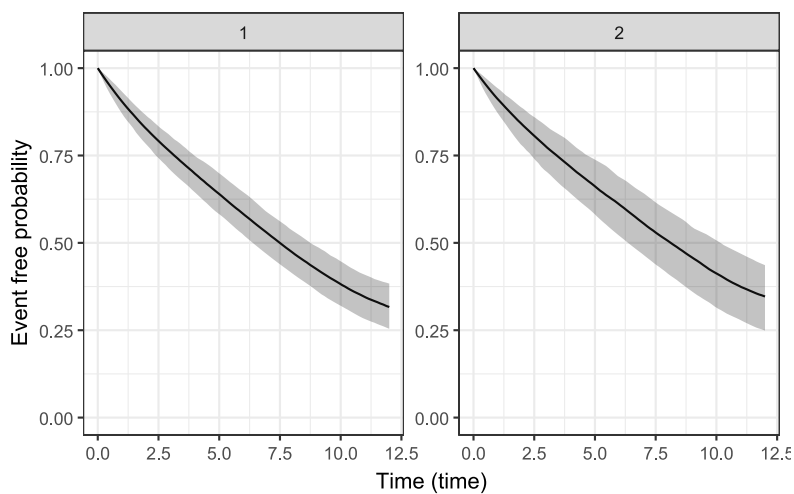
```
# label: time_to_switch_plots
ps_switch <- posterior_survfit(
  fit_surv_switch,
  newdata = data.frame(grp = c("SAT", "Control")),
  times = 0,
  extrapolate = TRUE,
  control = list(edist = 12))

panel_labels <- c('1' = "SAT", '2' = "Control")

plot(ps_switch) +
  ggplot2::facet_wrap(~ id, labeller = ggplot2::labeller(id = panel_labels))
```



```
plot(ps_switch)
```



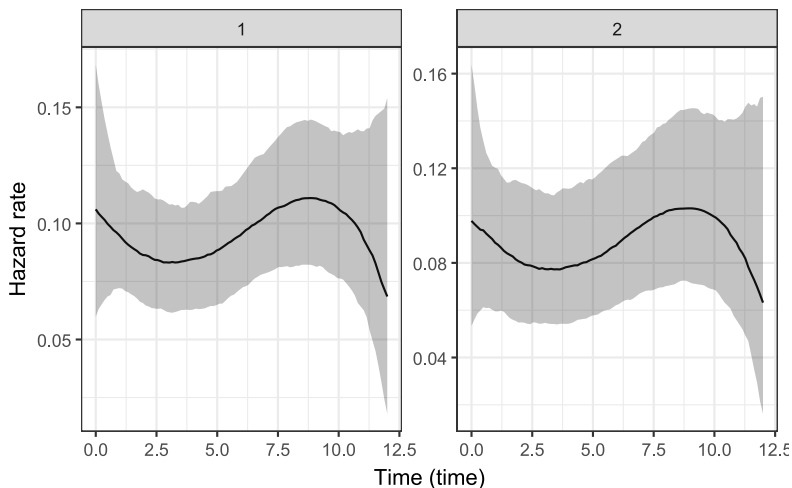
```
ph <- posterior_survfit(
  fit_surv_switch,
```

```

newdata = data.frame(grp = c("SAT", "Control")),
type = "haz"
)

plot(ph)

```



```

# --- Calculate median survival times and their difference ---
# We'd only do this if the median is actually reached within the follow-up time,
# otherwise it makes no sense.
# calculate median time to switch
ps_switch_matrix <- posterior_survfit(
  fit_surv_switch,
  newdata = data.frame(grp = factor(c("SAT", "Control"))),
  times = 0,
  extrapolate = TRUE,
  control = list(edist = 12),
  return_matrix = TRUE
)

time_points <- sapply(ps_switch_matrix, function(x) attr(x, "times")[1])
SAT_matrix <- sapply(ps_switch_matrix, function(x) x[, 1])
control_matrix <- sapply(ps_switch_matrix, function(x) x[, 2])

get_posterior_median_times <- function(posterior_matrix, time_points) {
  # For each row (posterior draw), find the time when survival drops <= 0.5
  median_times <- apply(posterior_matrix, 1, function(one_draw_surv_probs) {
    idx <- which(one_draw_surv_probs <= 0.5)[1]

    # If survival never drops to 0.5 for a draw return max follow-up time
    # This isn't quite correct, however, we expected that the VAST majority of patients
    # switch within 12 months, so this shouldn't happen
    if (is.na(idx)) {
      return(12) # Return maximum follow-up time if median not reached. Will switch to
      # extending prediction beyond 12 months if needed, or drop median calculation.
    } else {
      return(time_points[idx])
    }
  })
}

```

```

    return(median_times)
  }

SAT_medians <- get_posterior_median_times(SAT_matrix, time_points)
# Get the full posterior distribution of median times for the Control group
control_medians <- get_posterior_median_times(control_matrix, time_points)
# Calculate the posterior distribution of the difference (SAT - Control)
difference_in_medians <- SAT_medians - control_medians

group_medians_df <- data.frame(
  median_time = c(SAT_medians, control_medians),
  group = rep(c("SAT", "Control"),
              each = length(SAT_medians))
)
ggplot(group_medians_df, aes(x = median_time, y = group)) +
  ggdist::stat_halfeye() +
  labs(
    title = "Posterior Distributions of Median Time to Switch",
    x = "Median Time",
    y = "Group"
  ) +
  scale_x_continuous(n.breaks = 6) +
  theme(legend.position = "none")

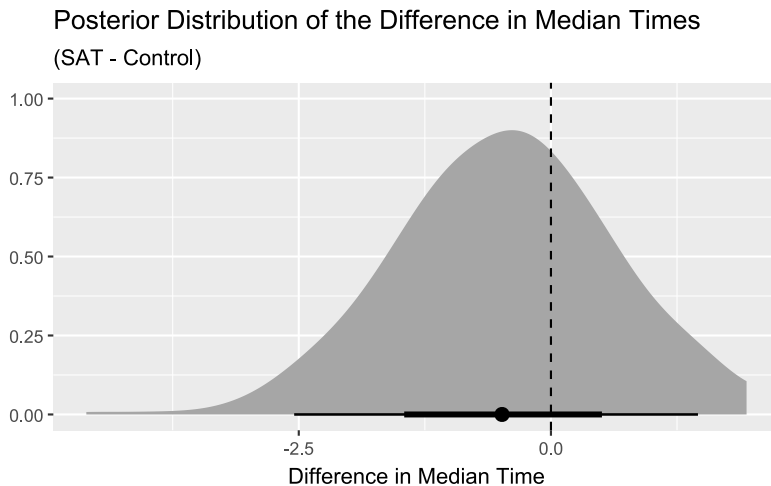
```



```

difference_df <- data.frame(difference = difference_in_medians)
ggplot(difference_df, aes(x = difference)) +
  ggdist::stat_halfeye() +
  geom_vline(xintercept = 0, linetype = "dashed") +
  labs(
    title = "Posterior Distribution of the Difference in Median Times",
    subtitle = "(SAT - Control)",
    x = "Difference in Median Time",
    y = NULL
  )

```

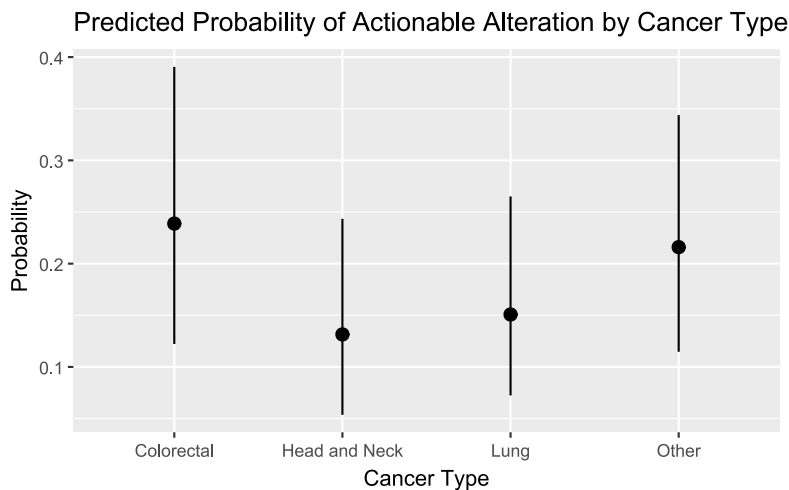


6. Actionable Alterations and Targeted Therapies

To (1) estimate the proportion of patients in the SAT arm with at least one actionable alteration identified via ctDNA analysis, and (2) compare the proportion of patients receiving targeted therapies between the SAT and external control groups.

```
# 6.1 Actionable Alterations
fit_actionable <- brm(
  actionable_alteration ~ 1 + cancer_type,
  data = sim_data |> filter(grp == "SAT"),
  family = bernoulli(),
  iter = 2000, chains = 4, cores = 1,
  seed = 123,
  file = "9-process-evaluation/models/fit_actionable.rds",
  file_refit = "on_change"
)

# Plot predictions (this is non-marginalized, just by cancer type (descriptive))
# We might also consider fitting cancer type as a random effect.
plot_predictions(fit_actionable, by = "cancer_type") +
  labs(
    title = "Predicted Probability of Actionable Alteration by Cancer Type",
    x = "Cancer Type",
    y = "Probability"
  )
```



```
# 6.2 Comparison of Targeted Therapies
```

```
fit_targeted <- brm(
  targeted_therapy ~ 1 + grp + age + cancer_type,
  data = sim_data |> filter(grp %in% c("SAT", "Control")),
  family = bernoulli(),
  iter = 2000, chains = 4, cores = 1,
  seed = 123,
  file = "9-process-evaluation/models/fit_targeted.rds",
  file_refit = "on_change"
)
```

Compiling Stan program...

Start sampling

SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 1).

Chain 1:

Chain 1: Gradient evaluation took 0.000109 seconds

Chain 1: 1000 transitions using 10 leapfrog steps per transition would take 1.09 seconds.

Chain 1: Adjust your expectations accordingly!

Chain 1:

Chain 1:

Chain 1: Iteration: 1 / 2000 [0%] (Warmup)

Chain 1: Iteration: 200 / 2000 [10%] (Warmup)

Chain 1: Iteration: 400 / 2000 [20%] (Warmup)

Chain 1: Iteration: 600 / 2000 [30%] (Warmup)

Chain 1: Iteration: 800 / 2000 [40%] (Warmup)

Chain 1: Iteration: 1000 / 2000 [50%] (Warmup)

Chain 1: Iteration: 1001 / 2000 [50%] (Sampling)

Chain 1: Iteration: 1200 / 2000 [60%] (Sampling)

Chain 1: Iteration: 1400 / 2000 [70%] (Sampling)

Chain 1: Iteration: 1600 / 2000 [80%] (Sampling)

Chain 1: Iteration: 1800 / 2000 [90%] (Sampling)

Chain 1: Iteration: 2000 / 2000 [100%] (Sampling)

```

Chain 1:
Chain 1: Elapsed Time: 0.49 seconds (Warm-up)
Chain 1:           0.327 seconds (Sampling)
Chain 1:           0.817 seconds (Total)
Chain 1:

SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 2).
Chain 2:
Chain 2: Gradient evaluation took 3.6e-05 seconds
Chain 2: 1000 transitions using 10 leapfrog steps per transition would take 0.36
seconds.
Chain 2: Adjust your expectations accordingly!
Chain 2:
Chain 2:
Chain 2: Iteration:    1 / 2000 [  0%] (Warmup)
Chain 2: Iteration:   200 / 2000 [ 10%] (Warmup)
Chain 2: Iteration:   400 / 2000 [ 20%] (Warmup)
Chain 2: Iteration:   600 / 2000 [ 30%] (Warmup)
Chain 2: Iteration:   800 / 2000 [ 40%] (Warmup)
Chain 2: Iteration:  1000 / 2000 [ 50%] (Warmup)
Chain 2: Iteration:  1001 / 2000 [ 50%] (Sampling)
Chain 2: Iteration:  1200 / 2000 [ 60%] (Sampling)
Chain 2: Iteration:  1400 / 2000 [ 70%] (Sampling)
Chain 2: Iteration:  1600 / 2000 [ 80%] (Sampling)
Chain 2: Iteration:  1800 / 2000 [ 90%] (Sampling)
Chain 2: Iteration:  2000 / 2000 [100%] (Sampling)
Chain 2:
Chain 2: Elapsed Time: 0.573 seconds (Warm-up)
Chain 2:           0.258 seconds (Sampling)
Chain 2:           0.831 seconds (Total)
Chain 2:

SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 3).
Chain 3:
Chain 3: Gradient evaluation took 3.4e-05 seconds
Chain 3: 1000 transitions using 10 leapfrog steps per transition would take 0.34
seconds.
Chain 3: Adjust your expectations accordingly!
Chain 3:
Chain 3:
Chain 3: Iteration:    1 / 2000 [  0%] (Warmup)
Chain 3: Iteration:   200 / 2000 [ 10%] (Warmup)
Chain 3: Iteration:   400 / 2000 [ 20%] (Warmup)
Chain 3: Iteration:   600 / 2000 [ 30%] (Warmup)
Chain 3: Iteration:   800 / 2000 [ 40%] (Warmup)
Chain 3: Iteration:  1000 / 2000 [ 50%] (Warmup)
Chain 3: Iteration:  1001 / 2000 [ 50%] (Sampling)
Chain 3: Iteration:  1200 / 2000 [ 60%] (Sampling)
Chain 3: Iteration:  1400 / 2000 [ 70%] (Sampling)
Chain 3: Iteration:  1600 / 2000 [ 80%] (Sampling)
Chain 3: Iteration:  1800 / 2000 [ 90%] (Sampling)
Chain 3: Iteration:  2000 / 2000 [100%] (Sampling)
Chain 3:
Chain 3: Elapsed Time: 0.562 seconds (Warm-up)
Chain 3:           0.29 seconds (Sampling)

```

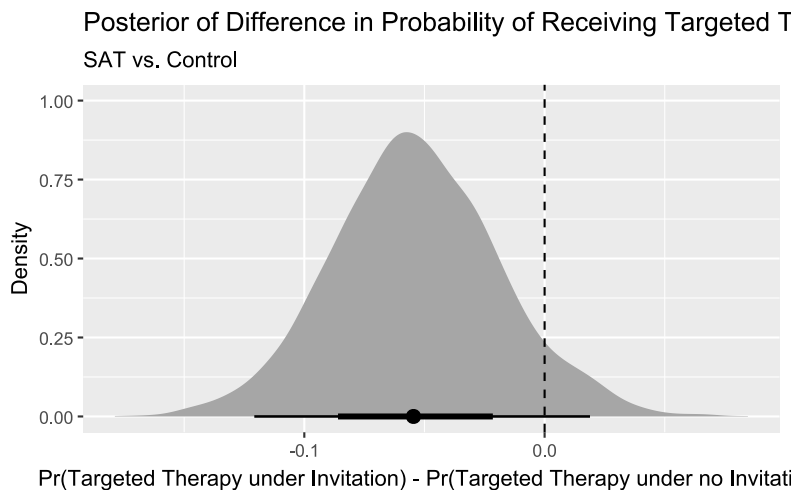


```
Chain 3:                0.852 seconds (Total)
Chain 3:

SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 4).
Chain 4:
Chain 4: Gradient evaluation took 3.6e-05 seconds
Chain 4: 1000 transitions using 10 leapfrog steps per transition would take 0.36
seconds.
Chain 4: Adjust your expectations accordingly!
Chain 4:
Chain 4:
Chain 4: Iteration:    1 / 2000 [  0%] (Warmup)
Chain 4: Iteration:   200 / 2000 [ 10%] (Warmup)
Chain 4: Iteration:   400 / 2000 [ 20%] (Warmup)
Chain 4: Iteration:   600 / 2000 [ 30%] (Warmup)
Chain 4: Iteration:   800 / 2000 [ 40%] (Warmup)
Chain 4: Iteration:  1000 / 2000 [ 50%] (Warmup)
Chain 4: Iteration:  1001 / 2000 [ 50%] (Sampling)
Chain 4: Iteration:  1200 / 2000 [ 60%] (Sampling)
Chain 4: Iteration:  1400 / 2000 [ 70%] (Sampling)
Chain 4: Iteration:  1600 / 2000 [ 80%] (Sampling)
Chain 4: Iteration:  1800 / 2000 [ 90%] (Sampling)
Chain 4: Iteration:  2000 / 2000 [100%] (Sampling)
Chain 4:
Chain 4: Elapsed Time: 0.513 seconds (Warm-up)
Chain 4:                0.29 seconds (Sampling)
Chain 4:                0.803 seconds (Total)
Chain 4:
```

```
# Counterfactual comparison (Risk Difference)
comp_targeted <- avg_comparisons(
  fit_targeted,
  variables = "grp",
  type = "response"
)

# Plot the posterior of the risk difference
comp_targeted |>
  posterior_draws() |>
  ggplot(aes(x = draw)) +
  stat_halfeye() +
  geom_vline(xintercept = 0, linetype = "dashed") +
  labs(
    title = "Posterior of Difference in Probability of Receiving Targeted Therapy",
    subtitle = "SAT vs. Control",
    x = "Pr(Targeted Therapy under Invitation) - Pr(Targeted Therapy under no
Invitation)",
    y = "Density"
  )
```



7. Blood Product Transfusions

To compare the rate of blood product transfusions (red blood cells or platelets) between the SAT and external control groups over the course of follow-up.

```
# here using time_to_bsc as proxy for follow-up time. We will assess it correctly for
# here in dummy data all patients have 12 months follow-up, but in real data this will
# vary

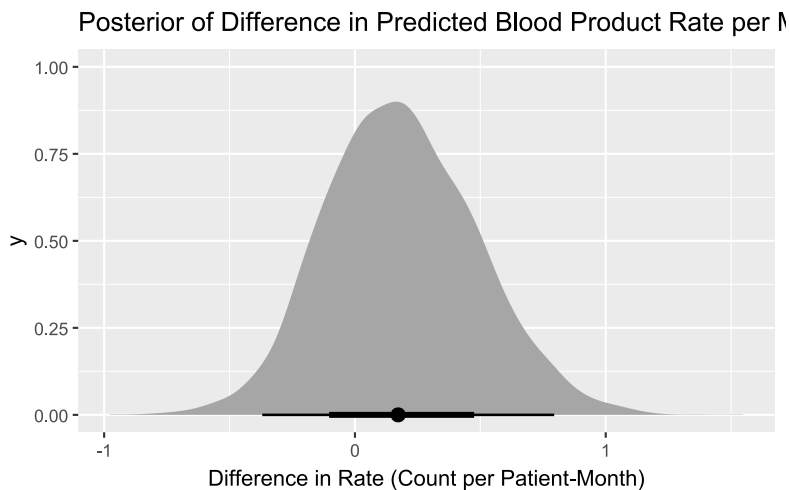
fit_blood <- brm(
  n_blood_products | rate(time_to_bsc) ~ 1 + grp + age + cancer_type,
  data = sim_data |> filter(grp %in% c("SAT", "Control")),
  family = negbinomial(),
  iter = 2000, chains = 4, cores = 4,
  seed = 123,
  file = "9-process-evaluation/models/fit_blood.rds",
  file_refit = "on_change"
)
```

Compiling Stan program...

Start sampling

```
# Compare predicted rates (marginalize over covariates for better precision)
comp_blood_rates <- avg_comparisons(fit_blood, variable = "grp", type = "response")

comp_blood_rates |>
  posterior_draws() |>
  ggplot(aes(x = draw)) +
  stat_halfeye() +
  labs(
    title = "Posterior of Difference in Predicted Blood Product Rate per Month",
    x = "Difference in Rate (Count per Patient-Month)",
  )
```



8. Time to Palliative Care

To compare the time from first-line treatment to the composite endpoint of a switch to best supportive care or death between the SAT and external control groups.

```
bsc_data <- surv_data |>
  mutate(time_to_bsc = pmin(time_to_bsc, follow_up_time),
         event_bsc = ifelse(time_to_bsc < follow_up_time, 1, 0))

fit_surv_bsc <- stan_surv(
  formula = Surv(time_to_bsc, event_bsc) ~ grp,
  data = bsc_data,
  basehaz = "ms",
  basehaz_ops = list(df = 5),
  iter = 2000,
  warmup = 1000,
  chains = 4,
  cores = 1,
  seed = 123
)
```

```
ps_pal <- posterior_survfit(
  fit_surv_bsc,
  newdata = data.frame(grp = c("SAT", "Control")),
  time = 0,
  control = list(edist = 12))

plot(ps_pal)
```

